

BRACU CP Workshop

Day 5

Ahnaf Shahriar Asif
Md Mubasshir Chowdhury

BRAC University, Dhaka

30 April 2025

Table of Contents

Introduction

Divisibility

Finding divisors of n

Primes

Problem solving

What will we learn ?

- ▶ Divisibility

What will we learn ?

- ▶ Divisibility
- ▶ Primes

What will we learn ?

- ▶ Divisibility
- ▶ Primes
- ▶ Modular Arithmetic

What will we learn ?

- ▶ Divisibility
- ▶ Primes
- ▶ Modular Arithmetic
- ▶ Primality testing

What will we learn ?

- ▶ Divisibility
- ▶ Primes
- ▶ Modular Arithmetic
- ▶ Primality testing
- ▶ Number theoretic functions

Divisibility

► Let a , b , and c be integers. Then:

- (i) If $b \mid a$ and $c \mid a$, then $bc \mid a$.
- (ii) If $b \mid a$ and $c \mid b$, then $c \mid a$.
- (iii) If $b \mid a$ and $c \mid b$, then $bc \mid ab$.
- (iv) If $c \mid a$ and $c \mid b$, then $c \mid (a + b)$ and $c \mid (a - b)$.

Finding divisors of n

- ▶ We can loop through all numbers and see if they divide n .

Finding divisors of n

- ▶ We can loop through all numbers and see if they divide n .
- ▶ Looping till $\sqrt{n}$ is enough.

Finding divisors of n

- ▶ We can loop through all numbers and see if they divide n .
- ▶ Looping till $\sqrt{n}$ is enough.
- ▶ We can find divisors of all numbers upto n in $\mathcal{O}(n \ln(n))$ time.

Finding divisors of n

```
vector<vector<int>> divisors(n + 1);  
for (int i = 1; i <= n; i++) {  
    for (int j = i; j <= n; j += i) {  
        divisors[j].push_back(i);  
    }  
}
```

Primes

- ▶ A prime p is greater than 1 and only divisible by 1 and p .

Primes

- ▶ A prime p is greater than 1 and only divisible by 1 and p .
- ▶ A composite number is a positive integer that is not prime.

Primes

- ▶ A prime p is greater than 1 and only divisible by 1 and p .
- ▶ A composite number is a positive integer that is not prime.
- ▶ Prime numbers help us calculate a lot of things easily.

Find number of divisors of n

- ▶ Given an integer $1 \leq n \leq 10^9$, find the number of divisors of n .

Solution: Number of divisors

- ▶ We can find the prime factorization of n .

Solution: Number of divisors

- ▶ We can find the prime factorization of n .
- ▶ If $n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$, then the number of divisors of n is given by:

$$d(n) = (e_1 + 1)(e_2 + 1) \cdots (e_k + 1)$$

Prime Factorization

- ▶ We can find the prime factorization of n in $\mathcal{O}(\sqrt{n})$ time.

Prime Factorization

- ▶ We can find the prime factorization of n in $\mathcal{O}(\sqrt{n})$ time.
- ▶ We can use a **sieve** to find all primes up to 10^6 in $\mathcal{O}(n \log(\log(n)))$ time.

Prime Factorization code

```
// vector<int> primes; (contains all primes upto sqrt(n))
vector<pair<int,int>> ppf(int n){
    vector<pair<int,int>> factors;
    for (int i = 0; primes[i] * primes[i] <= n; i++){
        int cnt = 0;
        while (n % primes[i] == 0){
            n /= primes[i];
            cnt++;
        }
        if (cnt > 0) factors.push_back({primes[i], cnt});
    }
    if (n > 1) factors.push_back({n, 1});
    return factors;
}

// if n = 12, it will return:
// {{2, 2}, {3, 1}} which means 12 = 2^2 * 3^1
```

Sieve

- ▶ We can use a sieve to find all primes up to 10^6 in $\mathcal{O}(n \log(\log(n)))$ time.

Sieve

- ▶ We can use a sieve to find all primes up to 10^6 in $\mathcal{O}(n \log(\log(n)))$ time.
- ▶ We can use the sieve to find the smallest prime factor of each number.

Sieve Simulation

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

Sieve Code

```
bool is_composite[N];
vector<int> primes;

void sieve(int n){
    is_composite[1] = true;
    for(int i = 2; i <= n; i++){
        if(!is_composite[i]){
            primes.push_back(i);
            for(int j = i * i; j <= n; j += i){
                is_composite[j] = true;
            }
        }
    }
}
```

Boxes Packing

Codeforces 903C

- ▶ You have n cube-shaped boxes, each with side length a_i

Boxes Packing

Codeforces 903C

- ▶ You have n cube-shaped boxes, each with side length a_i
- ▶ You can put box i into box j if $a_i < a_j$, box i is not already inside another box, and box j is empty

Boxes Packing

Codeforces 903C

- ▶ You have n cube-shaped boxes, each with side length a_i
- ▶ You can put box i into box j if $a_i < a_j$, box i is not already inside another box, and box j is empty
- ▶ Boxes can be nested multiple times

Boxes Packing

Codeforces 903C

- ▶ You have n cube-shaped boxes, each with side length a_i
- ▶ You can put box i into box j if $a_i < a_j$, box i is not already inside another box, and box j is empty
- ▶ Boxes can be nested multiple times
- ▶ A box is *visible* if it is not inside any other box. Your goal is to minimize the number of visible boxes

Boxes Packing

Codeforces 903C

- ▶ You have n cube-shaped boxes, each with side length a_i
- ▶ You can put box i into box j if $a_i < a_j$, box i is not already inside another box, and box j is empty
- ▶ Boxes can be nested multiple times
- ▶ A box is *visible* if it is not inside any other box. Your goal is to minimize the number of visible boxes
- ▶ **Input:** one integer n ($1 \leq n \leq 5000$), followed by a list of n integers a_1 to a_n ($1 \leq a_i \leq 10^9$)

Boxes Packing

Codeforces 903C

- ▶ You have n cube-shaped boxes, each with side length a_i
- ▶ You can put box i into box j if $a_i < a_j$, box i is not already inside another box, and box j is empty
- ▶ Boxes can be nested multiple times
- ▶ A box is *visible* if it is not inside any other box. Your goal is to minimize the number of visible boxes
- ▶ **Input:** one integer n ($1 \leq n \leq 5000$), followed by a list of n integers a_1 to a_n ($1 \leq a_i \leq 10^9$)
- ▶ **Output:** the minimum number of visible boxes

► Good Bye!